

CreditPathAI – Milestone 4 Reference Guide: Advanced Model Training (XGBoost & LightGBM)

1. Purpose of This Stage

This stage focuses on building high-performance models using XGBoost and LightGBM. Unlike Logistic Regression, these models capture non-linear relationships and interactions between variables. The goal is not just to train models, but to understand: • Why these models work • How each parameter affects behavior • How to systematically improve performance

2. Why Logistic Regression Was Not Enough

Logistic Regression assumes a linear relationship between features and output. In real loan datasets: • Risk is not linear • Multiple factors interact (income + loan amount) • Threshold effects exist (DTI > 0.5 suddenly risky) Tree-based models overcome this by splitting data into regions.

3. What is Boosting (Intuition)

Boosting builds models sequentially. Step 1: First tree makes predictions Step 2: Second tree focuses on mistakes Step 3: Third tree corrects remaining errors Final prediction = sum of all trees This allows capturing complex patterns gradually.

4. Load Data

`df = pd.read_csv("final_features.csv")` We use engineered features because: • Raw features are weak • Engineered features encode domain knowledge

5. Train-Test Split

```
from sklearn.model_selection import train_test_split
X = df.drop("default_status", axis=1)
y = df["default_status"]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

 Reason: Prevents overfitting by evaluating on unseen data.

6. XGBoost Model (Deep Explanation)

```
from xgboost import XGBClassifier
xgb = XGBClassifier(n_estimators=100, max_depth=5, learning_rate=0.1, subsample=0.8, colsample_bytree=0.8, random_state=42)
```

 Explanation:

n_estimators: Number of trees. Too low → underfitting Too high → overfitting max_depth: Controls tree complexity. Higher → more detailed splits but risk overfitting learning_rate: How much each tree contributes. Lower → safer but slower subsample: Fraction of data per tree. Prevents overfitting colsample_bytree: Fraction of features per tree. Adds randomness → improves generalization

7. LightGBM Model (Deep Explanation)

```
from lightgbm import LGBMClassifier lgbm = LGBMClassifier( n_estimators=100, max_depth=5,
learning_rate=0.1, subsample=0.8, colsample_bytree=0.8, random_state=42 )
```

 Key Difference: XGBoost → grows trees level-wise (balanced) LightGBM → grows leaf-wise (faster, deeper) Tradeoff: LightGBM is faster but can overfit more easily.

8. Predictions

```
y_prob_xgb = xgb.predict_proba(X_test)[:,-1] y_prob_lgbm = lgbm.predict_proba(X_test)[:,-1]
```

 We use probabilities instead of labels because: • AUC requires probability ranking

9. Evaluation (AUC-ROC)

```
from sklearn.metrics import roc_auc_score auc_xgb = roc_auc_score(y_test, y_prob_xgb) auc_lgbm =
roc_auc_score(y_test, y_prob_lgbm)
```

 Interpretation: 0.5 → random 0.7 → decent 0.8+ → strong Goal: Advanced models must outperform Logistic Regression baseline.

10. Hyperparameter Tuning (Detailed)

```
from sklearn.model_selection import GridSearchCV param_grid = { 'max_depth':[3,5,7],
'learning_rate':[0.01,0.1], 'n_estimators':[100,200] }
```

 grid = GridSearchCV(XGBClassifier(), param_grid, scoring='roc_auc', cv=3) grid.fit(X_train, y_train) Why GridSearch: • Systematically tests combinations • Avoids guesswork Drawback: • Computationally expensive

11. Alternative: Random Search

Instead of GridSearch: RandomizedSearchCV Advantage: Faster Drawback: May miss best combination

12. Feature Importance

```
importance = xgb.feature_importances_
```

 Important note: High importance ≠ causal relationship Use it to: • Validate features • Remove useless ones

13. Common Failure Cases

Case 1: AUC not improving → Weak features Case 2: Overfitting → High train AUC, low test AUC → Reduce depth, increase regularization Case 3: Slow training → Reduce estimators

14. Expected Results

Typical results: Logistic Regression → 0.70–0.75 XGBoost → 0.80–0.88 LightGBM → similar or slightly higher If not achieved: → revisit feature engineering

15. Final Deliverables

Students must submit: • XGBoost model • LightGBM model • AUC comparison • Hyperparameter tuning results